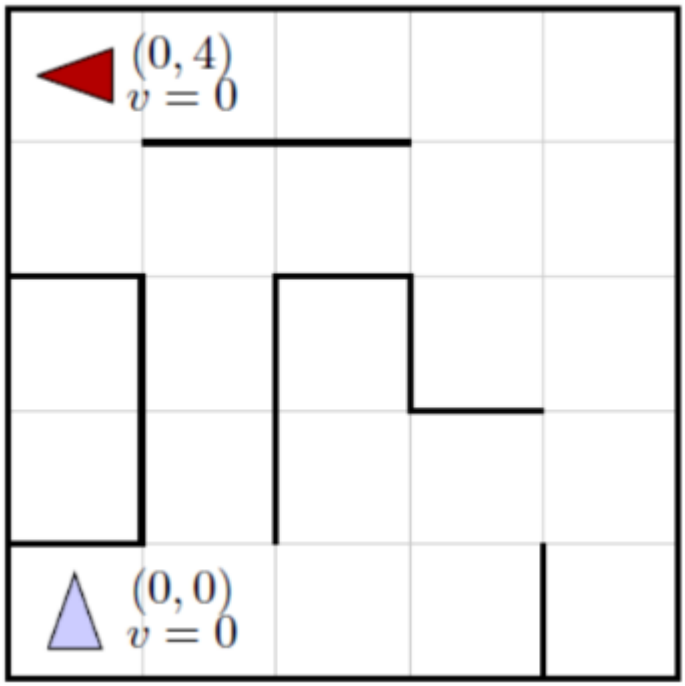
BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Họ và tên	Đỗ Xuân Triệu
MSSV / Lớp	3124720072

Câu 1. Tìm kiếm và heuristic



a) Chuỗi hành động ngắn nhất từ (0,0), Bắc -> (0,4), Tây (qua ô (1,2)):

Tác tử đi theo nhánh ngoài cùng bên trái (đi qua (1,2)). Chuỗi hành động gồm 16 bước như sau:

1. Tăng tốc → Đến (0,1), hướng Bắc, v = 1
2. Giữ tốc độ → Đến (0,2), hướng Bắc, v = 1
3. Giảm tốc → Dừng tại (0,2), hướng Bắc, v = 0
4. Rẽ phải → Tại (0,2), quay sang hướng Đông, v = 0
5. Tăng tốc → Đến (1,2), hướng Đông, v = 1
6. Giảm tốc → Dừng tại (1,2), hướng Đông, v = 0
7. Rẽ trái → Tại (1,2), quay sang hướng Bắc, v = 0
8. Tăng tốc → Đến (1,3), hướng Bắc, v = 1
9. Giảm tốc → Dừng tại (1,3), hướng Bắc, v = 0
10. Rẽ trái → Tại (1,3), quay sang hướng Tây, v = 0
11. Tăng tốc → Đến (0,3), hướng Tây, v = 1

12. Giảm tốc → Dừng tại (0,3), hướng Tây, v = 0
13. Rẽ phải → Tại (0,3), quay sang hướng Bắc, v = 0
14. Tăng tốc → Đến (0,4), hướng Bắc, v = 1
15. Giảm tốc → Dừng tại (0,4), hướng Bắc, v = 0
16. Rẽ trái → Dừng tại ô đích (0,4), hướng Tây, v = 0 (Hoàn thành!)

b) Kích thước không gian trạng thái:

Kích thước không gian trạng thái: $M \times N \times 4 \times (Vmax + 1)$

Giải thích: Một trạng thái hoàn chỉnh của tác tử được định nghĩa bởi 4 thành phần (x, y, d, v):

- Tọa độ x có M giá trị có thể có.
- Tọa độ y có N giá trị có thể có.
- Hướng d có 4 giá trị {Bắc, Nam, Đông, Tây}.
- Vận tốc v có (Vmax + 1) giá trị {0, 1, 2, ..., Vmax}.

c) Trạng thái không có nút con:

Lựa chọn: [X] Có [ ] Không

Giải thích / Ví dụ: Có tồn tại trạng thái không có nút con. Ví dụ: Xe đang di chuyển với vận tốc v > 0 và ở vị trí ngay sát trước một bức tường.
- Do v > 0 nên tác tử KHÔNG được phép rẽ.
- Nếu tác tử thực hiện hành động di chuyển (Tăng tốc, Giữ tốc độ, Giảm tốc đến v' > 0) thì xe sẽ đâm vào tường (hành động không hợp lệ).
Do đó, không có bất kỳ hành động nào hợp lệ có thể thực hiện, trạng thái này không sinh ra nút con nào.

d) Hệ số phân nhánh lớn nhất:

Hệ số phân nhánh lớn nhất: $b_{max} = 3$

Trạng thái ví dụ: (0, 0, Bắc, 0)

Các hành động có thể thực hiện: {Rẽ trái, Rẽ phải, Tăng tốc}. (Lưu ý: Giữ tốc độ khi v = 0 không hợp lệ).

e) Khoảng cách Manhattan có admissible không?

Lựa chọn: [ ] Có [X] Không

Phản ví dụ: Xét trường hợp Vmax >= 3. Giả sử tác tử đang ở vị trí (0,1), hướng Bắc, vận tốc v = 2. Ô đích nằm tại (0,4), hướng Bắc.
- Khoảng cách Manhattan = $|0 - 0| + |4 - 1| = 3$. Vậy h = 3.
- Tuy nhiên, với vận tốc v = 2, tác tử chỉ cần thực hiện 1 hành động Tăng tốc (v' = 3) để đi thẳng 3 ô đến đúng ô (0,4). Do đó chi phí thực tế tối ưu $h^* = 1$.
- Vì $h = 3 > h^* = 1$ (heuristic đánh giá cao hơn chi phí thực tế), nên khoảng cách Manhattan KHÔNG admissible.

f) Heuristic h = Manhattan / Vmax có admissible không?

Lựa chọn: [X] Có [ ] Không

Giải thích: Vận tốc tối đa trong 1 bước di chuyển là Vmax ô. Để di chuyển quãng đường có khoảng cách Manhattan là D, số bước di chuyển tối thiểu cần thiết luôn là D / Vmax. Do đó h(s) <= h*(s) với mọi trạng thái, đảm bảo tính admissible.

g) Ảnh hưởng đến tính đầy đủ (Completeness) trong A* Tree Search khi h không admissible:

Lựa chọn: [ ] Có [X] Không

Giải thích: Với đồ thị hữu hạn và chi phí không âm, A* Tree Search vẫn sẽ duyệt qua tất cả các trạng thái cho đến khi tìm thấy lời giải. Do đó tính đầy đủ KHÔNG bị thay đổi.

h) Ảnh hưởng đến tính tối ưu (Optimality) trong A* Tree Search khi h không admissible:

Lựa chọn: [X] Có [ ] Không

Giải thích: A* Tree Search chỉ đảm bảo tìm ra đường đi tối ưu khi h là admissible. Khi h không admissible (đánh giá cao hơn chi phí thực), thuật toán có thể cắt tỉa hoặc bỏ qua đường đi tối ưu và trả về kết quả không tối ưu.

i) Các lý do hợp lý khi dùng heuristic không admissible thay vì admissible:

Phương án được chọn:
[X] Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.
[X] Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.
[ ] Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.
[X] Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

Câu 2. Các nút tìm kiếm

Cho bài toán tìm kiếm cây với hệ số phân nhánh tối đa b. Gọi n_c là nút đích có chi phí nhỏ nhất và n_s là nút đích nông nhất.

a) Breadth-First Search (BFS):

Bất đẳng thức: $d(n) < d(n_s)$

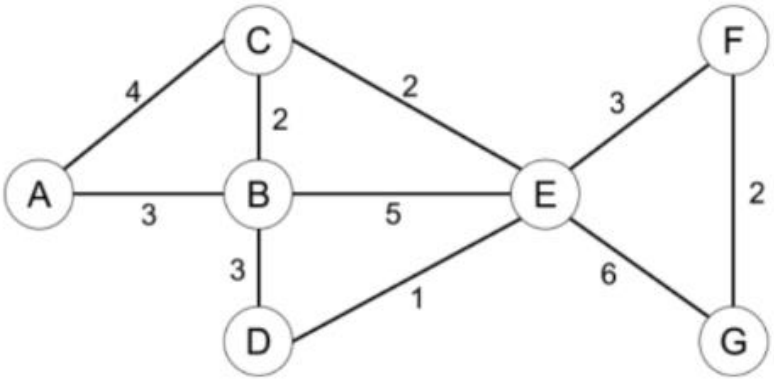
b) Uniform-Cost Search (UCS):

Bất đẳng thức: $g(n) < g(n_c)$

c) A* Search (giả sử h là heuristic nhất quán - consistent):

Bất đẳng thức: $g(n) + h(n) < g(n_c)$

Câu 3. Tìm kiếm trên đồ thị



Node	h_1	h_2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Nút	h1	h2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

a) Bảng kết quả đường đi của các thuật toán:

Thuật toán	A-C-E-G	A-C-E-F-G	A-B-D-E-F-G	Khác
UCS		**X**		
Greedy với h1				**X**
Greedy với h2	**X**			
A* với h1			**X**	
A* với h2		**X**		

b) Chi phí của đường đi tối ưu từ A đến G theo UCS:

Đáp án: 11 (Đường đi tối ưu là A - C - E - F - G với tổng chi phí: 4 + 2 + 3 + 2 = 11)

c) Đánh giá Heuristic h1:

Kết quả: Admissible: [ ] Có [X] Không | Consistent: [ ] Có [X] Không

Giải thích: Tại nút A, h1(A) = 12, nhưng chi phí thực tế tối ưu h*(A) = 11. Do 12 > 11 nên h1 không admissible (dẫn đến cũng không consistent).

d) Đánh giá Heuristic h2:

Kết quả: Admissible: [X] Có [ ] Không | Consistent: [ ] Có [X] Không

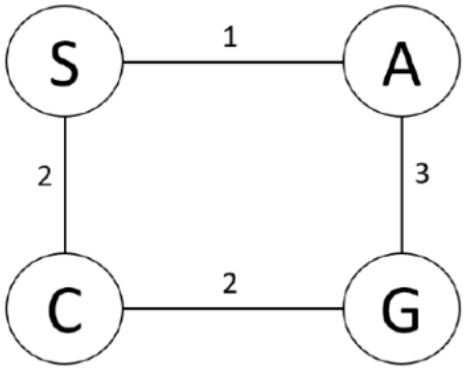
Giải thích: • Admissible: Mọi giá trị h2(n) đều nhỏ hơn hoặc bằng chi phí thực tế h*(n) đến G (chẳng hạn h2(A)=11 <= 11).
• Non-consistent: Xét cạnh (A, C) có chi phí c(A,C) = 4. Ta có h2(A) = 11, h2(C) = 6. Bất đẳng thức tam giác yêu cầu h2(A) <= c(A,C) + h2(C) <=> 11 <= 4 + 6 = 10 (Sai!). Do đó h2 không consistent.

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a) Đánh dấu Đúng / Sai:

Phát biểu	Đúng	Sai
Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.		**X**
Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.	**X**	
Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.	**X**	
Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.		**X**
Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.	**X**	
Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent.	**X**	

b) Điền các giá trị heuristic sao cho heuristic admissible nhưng không consistent:



Heuristic Function	
State	$h(s)$
S	
A	
C	
G	0

Trạng thái	h(s)
S	
A	
C	
G	0

Bảng giá trị heuristic cần điền:

Trạng thái (State)	**h(s)**
S	4
A	1
C	2
G	0

Chứng minh tính thỏa mãn:

1. Tính Admissible (Chấp nhận được):
   • Chi phí thực tế tối ưu từ các nút đến G là: h*(S) = 4, h*(A) = 3, h*(C) = 2, h*(G) = 0.
   • Giá trị h(s) đã chọn thỏa mãn h(s) <= h*(s) với mọi nút: h(S)=4 <= 4, h(A)=1 <= 3, h(C)=2 <= 2, h(G)=0 <= 0.
2. Tính Không Consistent (Không nhất quán):
   • Xét cạnh (S, A) có chi phí c(S, A) = 1.
   • Ta có: h(S) = 4, trong khi c(S, A) + h(A) = 1 + 1 = 2.
   • Do h(S) = 4 > c(S, A) + h(A) = 2, bất đẳng thức tam giác bị vi phạm. Do đó h(s) không consistent.